ProfileLink — Personalised QR Profile Pages

A production-ready system for generating personalised profile page links, scannable via QR codes.

Architecture

```
profilelink/
├── backend/          # Node.js + Express API server
│   ├── db/           # MySQL connection pool & schema
│   ├── routes/       # API routes (profiles, admin, auth)
│   ├── middleware/   # Security, rate limiting, auth
│   └── uploads/      # Profile photos (gitignored)
├── frontend/
│   ├── public/       # Static profile page (served at /:slug)
│   └── admin/        # Admin dashboard (React)
└── docker-compose.yml
```

Tech Stack

Layer	Technology	Reason
Runtime	Node.js 20 LTS + Express	Non-blocking I/O, handles massive concurrency
Database	MySQL 8 + connection pooling	Reliable, ACID, scales horizontally
Storage	Local disk (swap for S3 in prod)	Simple, fast; swap path in config
Frontend	Vanilla HTML/CSS/JS	Zero JS framework = instant load for viewers
Admin UI	Vanilla JS SPA	Lightweight, no build step needed
Security	Helmet, rate-limit, bcrypt, JWT	Defence-in-depth

Quick Start (Docker)

bash

```
# 1. Copy env
cp .env.example .env
# Edit .env — set DB_PASSWORD and JWT_SECRET

# 2. Start everything
docker-compose up -d

# 3. Create the first admin account
curl -X POST http://localhost:3000/api/auth/setup \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"YourStrongPassword123!"}'

# 4. Visit admin panel
open http://localhost:3000/admin
```

Manual Setup (without Docker)

Prerequisites

- Node.js 20+
- MySQL 8+

bash

```
# Install dependencies
cd backend && npm install

# Create DB
mysql -u root -p < db/schema.sql

# Configure env
cp .env.example .env   # fill in values

# Start server
npm start
```

Security Features

- **UUID-based profile slugs** — no sequential IDs to enumerate

- **Helmet.js** — sets 15+ security HTTP headers
- **Rate limiting** — 100 req/min on public routes, 10 req/min on admin login
- **Parameterised SQL queries** — zero SQL injection surface
- **bcrypt (12 rounds)** — admin password hashing
- **JWT (15min access + 7d refresh)** — stateless, short-lived admin sessions
- **File upload validation** — MIME type + magic bytes check, size limit, random filename
- **Input sanitisation** — DOMPurify-equivalent on all string fields
- **CORS** — locked to your domain
- **CSP headers** — strict Content-Security-Policy on profile pages
- **No sensitive data in URLs** — admin tokens in httpOnly cookies only

---

Profile URL Format

```
https://yourdomain.com/p/{uuid-v4}
```

Example: `https://yourco.com/p/a1b2c3d4-e5f6-7890-abcd-ef1234567890`

Generate QR code from this URL using any QR library or service.

API Reference

Public

Method	Path	Description
GET	`/p/:slug`	Serve profile page
GET	`/api/profile/:slug`	Get profile JSON

Admin (requires JWT)

Method	Path	Description
POST	`/api/auth/login`	Admin login
POST	`/api/auth/refresh`	Refresh token

Method	Path	Description
GET	`/api/admin/profiles`	List all profiles
POST	`/api/admin/profiles`	Create profile
PUT	`/api/admin/profiles/:id`	Update profile
DELETE	`/api/admin/profiles/:id`	Delete profile
POST	`/api/admin/profiles/:id/photo`	Upload photo

Setup (one-time)

Method	Path	Description
POST	`/api/auth/setup`	Create first admin (disabled after first use)